

S4. 개발환경 구성과 로컬 블록체인 포크 원리

Hardhat Mainnet Fork · 환경변수 · hardhat.config.ts · TypeChain

2026.05 | M1 S4

M1 — 오늘이 마지막 세션

S1

선행 과정 복습
개념 매핑
✓ 완료

S2

5레이어 아키텍처
당사 로드맵
✓ 완료

S3

스켈레톤 구조
의존 방향 규칙
✓ 완료

S4

개발환경 구성
로컬노드 + Sepolia
▶ 지금

S4 완료 = M1 완료. M2 DMZ 이벤트 파이프라인 진입 준비 완료

S4 — 오늘 배울 3가지

① 실습 환경 구분

- 로컬 노드
- Sepolia
- [맛보기] Mainnet Fork
- 언제 무엇을 쓰나

② 환경변수 설정

- .env 구조
- SEPOLIA_RPC_URL
- 보안 원칙

③ hardhat.config.ts

- 네트워크 설정
- 컴파일러 고정
- TypeChain 연결

3가지를 이해하면 실습 7단계가 자연스럽게 따라온다.

S4 완료 기준 — 3가지 달성

완료 기준 ①

`npx hardhat node`
→ 기동 성공
`localhost:8545`
계정 목록 출력 확인

완료 기준 ②

`npx hardhat compile`
→ 타입 에러 없음
`artifacts/` 생성

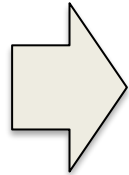
완료 기준 ③

`npx hardhat test`
→ skip/pending 목록
전체 실행 확인

개발환경의 딜레마 — 어디서 테스트하나

메인넷에서 직접 테스트?

- 실제 ETH 가스비 발생
- 실수 시 실제 자산 손실
- 배포 후 롤백 불가
- 속도 느림 (블록 ~12초)



로컬 환경에서 테스트!

- 무료 / 즉시 실행
- 실수해도 재시작 가능
- 메인넷 상태 그대로 복제
- 인스턴트 채굴

모든 블록체인 개발자는 로컬 환경에서 테스트한 뒤 메인넷에 배포한다.

Hardhat Network — 기본 로컬 체인

In-Process EVM

Hardhat이 내장한 가상 EVM 네트워크
테스트 전용 — 실제 블록체인 아님

특징

계정 20개 자동 생성 (10,000 ETH)
블록 즉시 채굴 / 가스비 무료

시작

`npm hardhat node` → `localhost:8545`

계정

`0xf39Fd6...` ~ 자동 생성 20개

가스

무료 (gas price 0 가능)

상태

재시작 시 초기화 (ephemeral)

용도

단위 테스트 / 빠른 기능 검증

로컬 체인의 한계 — 메인넷과 다르다

컨트랙트 없음	메인넷의 USDC·Uniswap·Aave 등 실제 컨트랙트 없음
상태 없음	실제 계정 잔액·NFT 보유 이력 없음 — 처음부터 배포 필요
가스 모델 단순	EIP-1559 baseFee 동적 변화 반영 안 됨
체인ID 고정	chainId=31337 — MetaMask 연동 시 주의
블록 타임 없음	즉시 채굴 → 실제 12초 블록 간격 재현 불가

실제 DeFi 프로토콜과 연동하거나 메인넷 상태가 필요할 때 → Fork 사용

Mainnet Fork — 메인넷을 로컬로 복사한다

Ethereum Mainnet

실제 컨트랙트
실제 계정 잔액
실제 블록 상태
블록 N번까지의 모든 기록

RPC
복제
→

Local Fork Node

메인넷 상태 그대로 복제
이후 트랜잭션은 로컬에서
메인넷에 기록 안 됨
가스비 무료 / 즉시 채굴

Fork = 특정 블록 시점의 메인넷 스냅샷을 로컬에서 그대로 쓰는 것

3가지 실습 환경 — 한눈에 비교

항목	로컬 노드	Sepolia	Mainnet Fork
인터넷 연결	불필요	필요 (RPC)	필요 (RPC)
체인 상태	완전 빈 체인	공개 테스트넷	메인넷 스냅샷 복제
ETH 비용	무료 (자동 생성)	무료 (faucet)	무료 (로컬)
블록 속도	즉시 채굴	~12초	즉시 채굴
기록 영구성	재시작 시 초기화	영구 기록	재시작 시 초기화
실제 컨트랙트	없음	일부만	메인넷 그대로
이 강의 사용	✓ M2~M5 단위 테스트	✓ 배포 전 검증	👁️ 맛보기 (M6~M7용)

이 강의 실습: 로컬 노드(개발·테스트) + Sepolia(배포 검증). Mainnet Fork는 개념만.

Fork가 필요한 실제 상황 3가지

① 실제 컨트랙트 연동

월렛원이 발행하는 NFT는
메인넷 컨트랙트 기반
→ Fork 없이 테스트 불가

② 잔액 기반 조건 판단

사용자 ETH / NFT 잔액으로
발행 조건 판단
→ 실제 잔액 필요

③ 업그레이드 안전성 검증

메인넷 배포 컨트랙트에
업그레이드 시뮬레이션
→ 실제 Storage Layout

M6~M7: 컨트랙트 배포·업그레이드·보안감사 — 전부 Fork에서 실행

Fork — 속도 vs 정확도 Trade-off

첫 실행 이후 캐시 적중
로컬 체인과 비슷한 속도

첫 실행 / 캐시 미스
RPC 조회로 느림 (수십 초)

속도 최적화 방법

- ① blockNumber 고정 — 같은 블록 기준으로 캐시 재활용
- ② Alchemy/Infura 전용 노드 — 공용 RPC보다 빠름
- ③ .hardhat/cache/ 유지 — 삭제하면 다시 느려짐
- ④ 단위 테스트는 HH Network — Fork는 통합 테스트에만

캐시 저장 위치

.hardhat/cache/server/<블록번호>/
<account>/<slot>.json

- 블록 번호 변경 시 캐시 무효화
- .gitignore에 .hardhat/ 추가 필수

blockNumber 고정하면 캐시 히트를 극대화 → 재실행 속도 로컬 수준

교보 프로젝트 — Fork 사용 전략

M1 ~ M5
오프체인 실습

HH Network
(기본 로컬 체인)
속도 최우선

M6 ~ M7
온체인 컨트랙트

Mainnet Fork
(Sepolia Fork)
정확도 필요

M8
Phase 1 통합

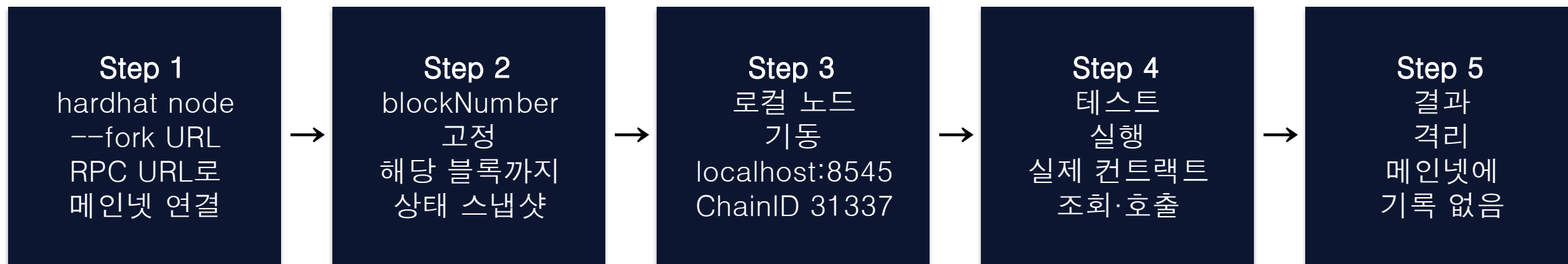
Fork + Stub
혼합 사용
E2E 검증

S4 실습은 Fork 기동 확인까지. 실제로 Fork 집중 사용하는 것은 M6부터.

M6~M7 — Fork에서 하는 것들

세션	주제	Fork에서 하는 것
M6 S39	컨트랙트 배포	Sepolia Fork에 KyoboNFT.sol 배포 → 주소 확인
M6 S40	UUPS 업그레이드	Fork에서 v1 → v2 업그레이드 → Storage 검증
M7 S41	보안 공격 재현	Fork에서 실제 재진입 공격 시뮬레이션
M7 S42	Slither 정적 분석	HIGH 취약점 발견 → 수정 → 재검증
M8 S50	Phase 1 통합	Fork에서 전체 흐름 장애 주입 + 복원력 확인

Fork 동작 플로우 — 순서



핵심: Fork는 특정 블록 시점의 스냅샷. 그 이후 변화는 로컬에서만 발생.

RPC URL — 메인넷 데이터 접근 경로

RPC Node (Alchemy / Infura)

블록체인 데이터를 HTTP로 제공하는 서비스
eth_getBalance · eth_call · eth_getCode



Hardhat Fork

RPC를 통해 필요한 데이터만 요청
캐시 후 로컬에서 EVM 실행

Alchemy 무료 플랜으로 충분

- 초당 요청 제한 있음 (무료: ~100 CU/s)
- 첫 Fork 기동 시 많은 요청 발생
- 이후 캐시로 해결
- 팀 전체가 같은 키 공유 시 한도 주의

RPC URL 형식

Alchemy Mainnet:

<https://eth-mainnet.g.alchemy.com/v2/KEY>

Alchemy Sepolia:

<https://eth-sepolia.g.alchemy.com/v2/KEY>

Infura: <https://mainnet.infura.io/v3/KEY>

Alchemy 무료 계정 → API Key 발급 → .env에 저장. 절대 커밋 금지.

forking.blockNumber — 블록 번호 고정

blockNumber 고정 시

매번 같은 블록 기준 실행
캐시 완전 활용
재현 가능한 테스트



blockNumber 미고정 시

최신 블록 기준 → 매번 변경
캐시 재활용 불가
테스트 결과 비재현

hardhat.config.ts 설정

```
networks: {  
  hardhat: {  
    forking: {  
      url: process.env.MAINNET_FORK_URL,  
      blockNumber: 21_500_000, // 고정  
    }  
  }  
}
```

블록 번호 선택 기준

- 대상 컨트랙트 배포 이후 블록
- 팀 전체가 동일 번호 사용
- 변경 시 → 팀 캐시 일괄 삭제
- CI/CD와 로컬 동일 번호 유지

블록 번호 = Fork의 타임스탬프. 고정 않으면 테스트가 날마다 다른 결과.

블록 번호 고정 — 왜 중요한가

고정 안 하면 — 어제 통과한 테스트가 오늘 실패한다

문제 시나리오

어제: 블록 21,000,000 기준 → 테스트 통과
오늘: 블록 21,100,000 기준 실행
→ 대상 주소의 ETH 잔액 달라짐
→ 조건 판단 결과 변경 → 실패
→ 코드는 안 바뀌었는데?

고정 후

팀 전원 21,500,000번 기준
→ 동일한 상태에서 실행
→ 어제와 오늘도 같은 결과
→ CI/CD에서도 동일하게 통과
→ '환경 차이' 버그 사라짐

blockNumber 고정 = 테스트 재현성 보장. CI/CD에서 필수.

impersonateAccount — 메인넷 계정을 로컬에서 사용

메인넷의 대형 계정을
로컬에서 그대로 사용 가능
실제 잔액 · 권한 그대로

교보 프로젝트 활용 예시
컨트랙트 Owner 계정 탈취
→ 업그레이드·권한 테스트

사용 방법

```
await network.provider.request({
  method: "hardhat_impersonateAccount",
  params: ["0xf3...대형계정주소"],
});
const whale = await ethers.getSigner("0xf3...");
// 이후 whale.sendTransaction() → 실제 잔액으로 실행
```

M6~M7: 컨트랙트 Owner 권한 테스트 시 활용. Fork 없이는 불가.

Hardhat 네트워크 4가지 — 언제 어느 것을 쓰나

네트워크	ChainID	속도	용도
hardhat (In-Process)	31337	⚡ 가장 빠름	단위 테스트 M2~M5
localhost	31337	⚡ 빠름	수동 테스트·스크립트
hardhat --fork	31337	🟡 첫 실행 느림	통합 테스트 M6~M7
sepolia	11155111	🔴 느림 (~12초)	실제 테스트넷 배포

교보: M2~M5 = In-Process / M6~M7 = Fork / 최종 배포 = Sepolia

모듈별 실습 네트워크 — 전체 매핑

M	세션	주제	실습 네트워크
M1	S1~S4	환경 셋업	In-Process
M2	S5~S12	DMZ 파이프라인	In-Process
M3	S13~S22	VASP 추상화	In-Process
M4	S23~S26	원장 + 감사로그	In-Process
M5	S27~S34	비즈니스 로직	In-Process
M6	S35~S40	ERC-1155 컨트랙트	Mainnet Fork
M7	S41~S44	보안감사 + 업그레이드	Mainnet Fork
M8	S45~S50	키 거버넌스 + 통합	Fork + Sepolia

Fork 캐시 — 두 번째 실행부터 빠르다

첫 실행 RPC 조회 발생

블록 헤더·계정·컨트랙트 코드·Storage 조회
느림 (수십 초)

캐시 경로

```
.hardhat/cache/server/  
<chainId>/<blockNumber>/  
<address>/<slot>.json
```

두 번째 실행 캐시 히트

.hardhat/cache/ 에서 즉시 읽음
로컬 체인과 비슷한 속도

캐시 전략

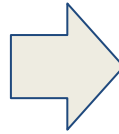
- blockNumber 고정 → 캐시 완전 활용
- blockNumber 변경 → 캐시 전체 재구성
- .gitignore에 .hardhat/ 추가 필수
- 팀원 각자 캐시 독립적으로 구성

캐시 삭제: `rm -rf .hardhat/cache` — 느려지면 먼저 시도

환경변수 — 왜 코드에 URL을 직접 넣으면 안 되나

❌ 하드코딩

코드 안에 API 키 직접 입력
GitHub 커밋 → 영구 노출
키 무효화 + 재발급 + 전체 재배포



✅ 환경변수

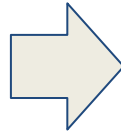
.env 파일에 분리 보관
.gitignore로 커밋 차단
환경별 다른 값 주입 가능

GitHub에 API 키 노출 = 봇이 수분 내 탐지 → 수천 달러 청구서 도착

.env vs .env.example — 역할 분리

.env (Git 제외)

실제 API 키 포함
.gitignore에 반드시 추가
개발자 로컬에만 존재
팀 공유 절대 금지



.env.example (Git 포함)

키 자리에 placeholder
신규 팀원 온보딩 가이드
어떤 변수가 필요한지 명시
실제 값은 별도 안전 채널로

.env는 커밋 금지 / .env.example은 커밋 필수 — 반드시 구분

.env 파일 구조 — 교보 프로젝트

.env (실제 값 — 커밋 금지)

```
# RPC URLs
MAINNET_FORK_URL=https://eth-mainnet.g.alche
my.com/v2/실제키
SEPOLIA_RPC_URL=https://eth-sepolia.g.alchemy.
com/v2/실제키

# 배포용 Private Key
DEPLOYER_PRIVATE_KEY=0x실제키

# 내부망 연결
INTERNAL_GATEWAY_SECRET=실제시크릿
INTERNAL_GATEWAY_URL=http://localhost:8080

# Sepolia 검증
ETHERSCAN_API_KEY=실제키
```

.env.example (placeholder — 커밋 O)

```
# RPC URLs
MAINNET_FORK_URL=https://eth-mainnet.g.alche
my.com/v2/YOUR_KEY
SEPOLIA_RPC_URL=https://eth-sepolia.g.alchemy.
com/v2/YOUR_KEY

# 배포용 Private Key
DEPLOYER_PRIVATE_KEY=0xYOUR_KEY

# 내부망 연결
INTERNAL_GATEWAY_SECRET=your-secret
INTERNAL_GATEWAY_URL=http://localhost:8080

# Sepolia 검증
ETHERSCAN_API_KEY=YOUR_KEY
```


Alchemy vs Infura — 교보 프로젝트 선택

항목	Alchemy	Infura
무료 한도	300M CU/월	100,000 req/일
응답 속도	빠름 (CDN)	보통
디버그 도구	Mempool Watch	제한적
교보 추천	✓ 권장	대안
Fork 안정성	매우 안정적	안정적
Sepolia	✓	✓

실습용: Alchemy 무료 계정으로 충분.

RPC URL 보안 — 절대 지켜야 할 3원칙

❌ 절대 금지

코드에 직접 작성
Slack / Discord 공유
GitHub 커밋
스크린샷에 노출

⚠️ 주의

팀원 공유 시
1Password 사용
환경별 키 분리
(로컬/스테이징/운영)
정기 로테이션

✅ 올바른 방법

.env 파일에 저장
.gitignore에 .env 추가
신규 팀원은
.env.example 참조

키 노출 시: Alchemy 대시보드 → API Key → Regenerate (즉시 무효화)

dotenv — .env 파일을 Node.js에 로드

hardhat.config.ts 최상단

```
import * as dotenv from "dotenv";  
dotenv.config();  
  
// 이후 process.env.MAINNET_FORK_URL 사용 가능
```

순서 주의

dotenv.config()는 반드시
process.env 접근 전에 호출.
hardhat.config.ts 맨 위에 위치.

dotenv.config() 한 줄이 .env 파일 전체를 process.env에 로드

설치 확인

npm install 시 자동 설치됨. package.json에 이미 포함.

환경변수 미설정 — 어디서 어떻게 실패하나

MAINNET_FORK_URL 미설정

```
process.env.MAINNET_FORK_URL = undefined  
forking.url = undefined → 에러
```

에러 메시지

```
Error: HH604: Error connecting to the network  
Cannot connect to undefined URL
```

```
또는  
HardhatError: Invalid url: undefined
```

방어적 설정

```
const FORK_URL =  
process.env.MAINNET_FORK_URL;  
if (!FORK_URL) {  
  throw new Error(  
    "MAINNET_FORK_URL is not set.\n"  
    + "Copy .env.example to .env"  
  );  
}
```

→ 의미 있는 에러 메시지로 빠른 원인 파악 가능

에러 시 체크: ① .env 파일 존재 ② 변수명 오타 ③ dotenv.config() 위치

환경변수 설정 체크리스트

실습 시작 전 이 순서대로 확인

- 1 Alchemy 계정 생성 → API Key 발급
- 2 .env.example 복사 → .env 생성
- 3 MAINNET_FORK_URL에 실제 Alchemy URL 입력
- 4 .gitignore 확인 — .env 항목 있는지
- 5 hardhat.config.ts 상단 dotenv.config() 있는지
- 6 터미널에서 URL 값 출력 확인

hardhat.config.ts — Hardhat의 모든 설정이 여기에

네트워크 설정

어느 체인에 연결?
Fork URL은?
Sepolia 연결 정보

컴파일러 설정

Solidity 버전
최적화 옵션
EVM Version

플러그인 설정

hardhat-toolbox
typechain 경로
etherscan 검증

이 파일 하나가 Hardhat 전체 동작을 제어한다.

hardhat.config.ts — 전체 구조

전체 구조

```
import "@nomicfoundation/hardhat-toolbox";
import * as dotenv from "dotenv";
dotenv.config();

const config: HardhatUserConfig = {
  solidity: { ... },    // 컴파일러
  networks: { ... },    // 네트워크
  typechain: { ... },   // 타입 생성
  etherscan: { ... },   // 검증
};

export default config;
```

각 섹션 역할

solidity

- Solidity 컴파일러 버전 고정
- 최적화 설정

networks

- hardhat(로컬) / localhost / sepolia
- forking 설정 (Fork URL + blockNumber)

typechain

- 컴파일 후 TS 타입 자동 생성
- outDir: 'typechain-types'

etherscan

- Sepolia 배포 후 소스코드 검증

networks.hardhat — 기본 로컬 체인 설정

forking 미설정 → 순수 In-Process 로컬 체인 →
M2~M5 단위 테스트

forking 없음 (M2~M5)

```
networks: {  
  hardhat: {  
    chainId: 31337,  
    mining: {  
      auto: true,  
      interval: 0,  
    },  
  },  
}
```

forking 설정 시 → Mainnet Fork 모드 전환 →
M6~M7 통합 테스트

forking 있음 (M6~M7)

```
networks: {  
  hardhat: {  
    forking: {  
      url: process.env.MAINNET_FORK_URL!,  
      blockNumber: 21_500_000,  
    },  
    chainId: 31337,  
  },  
}
```

forking 블록만 추가하면 로컬 체인 → Fork 모드 전환.

networks.localhost — 외부 프로세스 연결

hardhat node 별도 실행 후
다른 터미널에서 접속
localhost:8545

언제 쓰나?
수동 트랜잭션 전송
스크립트 실행 / Remix 연결

설정

```
networks: {  
  localhost: {  
    url: "http://127.0.0.1:8545",  
    chainId: 31337,  
  },  
}
```

실행 순서

터미널 1: npx hardhat node
터미널 2: npx hardhat run scripts/xxx.ts
--network localhost

localhost와 hardhat In-Process는 같은 ChainID지만 다른 프로세스.

networks.sepolia — 실제 테스트넷 연결

공개 테스트넷

실제 블록 생성 ~12초

Sepolia ETH 필요 (Faucet)

설정

```
networks: {
  sepolia: {
    url: process.env.SEPOLIA_RPC_URL!,
    chainId: 11155111,
    accounts: [
      process.env.DEPLOYER_PRIVATE_KEY!,
    ],
  },
}
```

언제 쓰나?

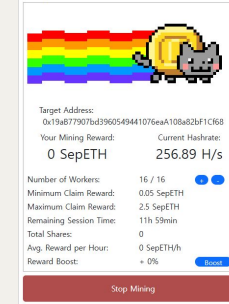
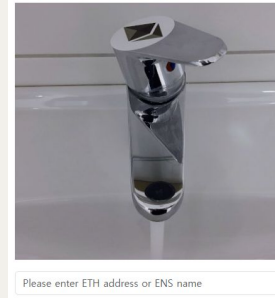
최종 배포 전 실제 환경 검증

Etherscan 소스코드 공개

Faucet — Sepolia ETH 받기

Sepolia PoW

Sepolia PoW Faucet — sepolia-faucet.com/



→ M6 배포 실습 전 미리 받기

Sepolia 배 포는 M6~M8에서만. M2~M5는 Sepolia 불필요.

forking 설정 — 3가지 필드

필드	역할	권장값
url	메인넷 RPC 엔드포인트	Alchemy Mainnet URL
blockNumber	Fork 기준 블록 번호	21_500_000 (팀 합의)
enabled	Fork 모드 켜기/끄기	true (url 있으면 자동 true)

완성된 forking 설정

```
forking: { url: process.env.MAINNET_FORK_URL!, blockNumber: 21_500_000 }
```

solidity — 컴파일러 설정

단일 버전 설정

```
solidity: {  
  version: "0.8.28",  
  settings: {  
    optimizer: {  
      enabled: true,  
      runs: 200,  
    },  
    evmVersion: "cancun",  
  },  
}
```

optimizer.runs 의미

runs: 200
= 배포 비용과 실행 비용의 균형점

자주 호출되는 함수 → 높게
배포 비용 낮추려면 → 낮게

교보 프로젝트: 200 (기본값)

교보 프로젝트: 0.8.28 단일 버전 고정. 팀 전체 동일 버전 보장.

evmVersion: cancun

최신 EVM 명령어 지원. PUSH0 (EIP-3855) 포함. 구버전 체인 배포 시 변경 필요.

Solidity 버전 고정 — 왜 중요한가

버전 미고정 → 빌드 환경마다 다른 컴파일러 → 다른 바이트코드 → 다른 배포 주소

버전 불일치 시나리오

개발자 A: 0.8.28로 컴파일
개발자 B: 0.8.25로 컴파일
→ 동일 소스지만 바이트코드 다름
→ 배포 시 주소 불일치
→ Etherscan 검증 실패

버전 고정 효과

hardhat.config.ts에 0.8.28 고정
→ 팀 전체 동일 컴파일러
→ 동일 바이트코드 보장
→ 배포 주소 재현 가능
→ Etherscan 검증 성공

Solidity는 마이너 버전도 바이트코드가 달라진다. 패치 버전까지 정확히 고정.

TypeChain — 컨트랙트 ABI → TypeScript 타입 자동 생성

ABI (Application Binary Interface)

컨트랙트 함수·이벤트 설명서
컴파일 시 artifacts/ 에 자동 생성



TypeChain

ABI를 읽어 TypeScript 타입 자동 생성
typechain-types/ 폴더에 저장

typechain 설정

```
typechain: {  
  outDir: "typechain-types",  
  target: "ethers-v6",  
}
```

import 방법

```
import { KyoboNFT__factory }  
  from "../typechain-types";  
  
const nft = KyoboNFT__factory  
  .connect(address, signer);
```

`npx hardhat compile → artifacts/ + typechain-types/` 동시 생성

네트워크 설정 요약 — 4개 비교

설정키	hardhat	localhost	sepolia	hardhat+fork
url	내장	http://127.0.0.1:8545	SEPOLIA_RPC_URL	MAINNET_FORK_URL
chainId	31337	31337	11155111	31337
accounts	자동 20개	자동 20개	DEPLOYER_KEY	자동 20개
forking	없음	없음	없음	blockNumber 고정
mining	auto=true	auto=true	12초/블록	auto=true

실습 — 7단계 전체 흐름



Step 4·5 = Fork 기동 확인 / Step 6 = TypeChain 생성 / Step 7 = 전체 파악

Step 1 — 의존성 설치 확인

Step 1

의존성 설치 확인

실행

```
# 레포 루트에서  
npm install
```

```
# 확인
```

```
ls  
node_modules/@nomicfoundation/hardhat-toolbox  
ls node_modules/typechain
```

주요 의존성

```
@nomicfoundation/hardhat-toolbox  
→ hardhat + ethers + typechain + mocha
```

```
@openzeppelin/contracts  
→ ERC-1155, UUPS, AccessControl
```

완료 기준: npm install 에러 없음 + node_modules/ 존재

Step 2 — .env 파일 생성

Step 2

.env 파일 생성

실행

```
# blockchain/ 폴더에서  
cp .env.example .env  
  
# .env 열고 값 입력  
SEPOLIA_RPC_URL=https://eth-sepolia.g.alchemy.  
com/v2/YOUR_KEY  
DEPLOYER_PRIVATE_KEY=0xYOUR_TEST_KEY
```

확인

```
node -e "  
require('dotenv').config();  
console.log(  
  process.env.SEPOLIA_RPC_URL  
);"
```

→ https://eth-sepolia... 출력되면 OK

완료 기준: 터미널에서 SEPOLIA_RPC_URL 값 출력 확인

Step 3 — hardhat.config.ts 구성

Step 3

hardhat.config.ts 구성

핵심 설정 확인 — 실습 환경 기본값 (forking 없음)

```
import "@nomicfoundation/hardhat-toolbox";
import * as dotenv from "dotenv";
dotenv.config();

const config: HardhatUserConfig = {
  solidity: { version: "0.8.28", settings: { optimizer: { enabled: true, runs: 200 } } },
  networks: {
    hardhat: { chainId: 31337 }, // forking 없음 → 빈 로컬 체인 (실습 기본)
    sepolia: { url: process.env.SEPOLIA_RPC_URL ?? "", chainId: 11155111, accounts: [...]} ,
    // forking 맞보기: hardhat: { forking: { url: MAINNET_RPC_URL, blockNumber: 21_500_000 } }
  },
};
```

완료 기준: 저장 후 IDE 문법 에러 없음

Step 4 — 로컬 노드 기동

Step 4

Hardhat 로컬 노드 기동 — 핵심 실습

실행 (fork 없음 — 기본)

```
# blockchain/ 폴더에서  
npx hardhat node
```

```
# [맞보기] Mainnet Fork 기동 시  
# npx hardhat node W  
# --fork  
https://eth-mainnet.g.alchemy.com/v2/KEY W  
# --fork-block-number 21500000
```

기동 성공 시 출력

```
Started HTTP and WebSocket JSON-RPC server  
at http://127.0.0.1:8545/
```

```
Accounts  
=====  
(0) 0xf39Fd6e51aad88... (10000 ETH)  
...  
(19) 0x... (10000 ETH)
```

완료 기준: 'Started HTTP and WebSocket JSON-RPC server' 메시지 출력

Step 5 — 블록 번호 확인

Step 5

Fork 블록 번호 확인 — Fork 정상 동작 검증

새 터미널에서 확인

```
node -e "  
const { ethers } = require('ethers');  
const p = new  
ethers.JsonRpcProvider('http://127.0.0.1:8545');  
p.getBlockNumber().then(console.log);"
```

출력 예시

21500000

- 설정한 blockNumber와 일치
- Fork 정상 동작 확인

다른 번호 출력 시:
→ config.ts forking 설정 재확인

완료 기준: 21500000 출력 (config에 설정한 blockNumber와 일치)

Step 6 — 스켈레톤 컴파일

Step 6

컴파일 → TypeChain 타입 생성 확인

실행

```
# blockchain/ 폴더에서  
npx hardhat compile
```

```
# 성공 시  
Compiled N Solidity files successfully
```

```
# 타입 생성 확인  
ls typechain-types/
```

생성되는 파일

```
typechain-types/  
  KyoboNFT.ts      ← 타입  
  factories/  
    KyoboNFT__factory.ts ← 배포 팩토리
```

```
artifacts/  
  contracts/KyoboNFT.sol/  
    KyoboNFT.json ← ABI
```

완료 기준: 에러 없이 컴파일 완료 + typechain-types/ 폴더 생성

Step 7 — 전체 테스트 실행

Step 7

skip/pending 목록 파악 — 앞으로 채울 것 확인

실행

```
# blockchain/ 폴더에서  
npx hardhat test
```

결과 예시

M1 Architecture

- ✓ S1: prior course check
- S5: Redis Streams (pending)

...

5 passing, 47 pending

pending vs failing

pending (회색 dash)

→ 정상. 아직 구현 전 stub

→ M2~M8에서 순서대로 채울 것

failing (빨간 X)

→ 환경 문제 → 즉시 해결

완료 기준: 빨간 failing 없음. pending은 정상.

실습 자주 하는 실수 ① — RPC URL 노출

⚠️ GitHub에 API 키 노출 = 수분 내 봇 탐지 → 수천 달러 청구

노출되는 경로

- ① hardhat.config.ts에 직접 입력 후 커밋
- ② .env 파일 자체를 커밋
- ③ CI 로그에 URL 출력
- ④ 스크린샷에 터미널 URL 포함

발생 시 대응

- ① Alchemy 대시보드 즉시 접속
- ② API Key → Regenerate
- ③ 새 Key를 .env에 업데이트
- ④ git history에서 파일 제거

예방이 최선: git add 전 git diff로 .env 포함 여부 확인

실습 자주 하는 실수 ② — 블록 번호 미고정

blockNumber 생략 시

매 실행마다 최신 블록
테스트 결과 비재현
팀원마다 다른 결과

blockNumber 고정 시

항상 같은 블록 기준
캐시 완전 활용
CI/CD에서도 동일

잘못된 설정

```
forking: {  
  url: process.env.MAINNET_FORK_URL!,  
  // blockNumber 생략 → 매번 최신  
}
```

올바른 설정

```
forking: {  
  url: process.env.MAINNET_FORK_URL!,  
  blockNumber: 21_500_000, // 고정  
}
```

blockNumber 한 줄이 캐시·재현성·팀 동기화 모두를 해결한다.

TypeChain — 왜 필요한가

TypeChain 없이

`contract.mintBatch(...any[ ])`
오타·타입 불일치 → 런타임 에러
IDE 자동완성 없음

TypeChain 있으면

`contract.mintBatch(to, ids, amounts, data)`
파라미터 타입 즉시 검증
IDE 자동완성 완벽

ABI를 TypeScript 타입으로 변환 = 컨트랙트를 타입 안전하게 사용

ABI → TypeScript 타입 생성 흐름



핵심

npx hardhat compile 한 번 → artifacts/ + typechain-types/ 동시 생성
Solidity 변경 → 재컴파일 → typechain-types 자동 갱신 → ABI 변경 시 컴파일 에러

typechain-types — 생성되는 파일 구조

폴더 구조

```
typechain-types/  
  index.ts          ← 전체 export  
  KyoboNFT.ts       ← 타입 인터페이스  
  NFTIssuer.ts  
  ActivityOracle.ts  
  common.ts         ← 공통 타입  
  factories/  
    KyoboNFT__factory.ts ← 배포 팩토리  
    NFTIssuer__factory.ts
```

KyoboNFT.ts (자동 생성 예시)

```
export interface KyoboNFT {  
  mintBatch(  
    to: string,  
    ids: BigNumberish[],  
    amounts: BigNumberish[],  
    data: BytesLike  
  ): Promise<ContractTransactionResponse>;  
  
  balanceOf(  
    account: string,  
    id: BigNumberish  
  ): Promise<bigint>;  
}
```

이 파일은 직접 수정 금지. compile 시 덮어써짐.

TypeChain 타입 — import해서 사용하는 방법

Factory로 배포

```
import { KyoboNFT__factory }  
  from "../typechain-types";  
  
const factory = new KyoboNFT__factory(signer);  
const nft = await factory.deploy();  
await nft.waitForDeployment();  
console.log(await nft.getAddress());
```

기존 컨트랙트 연결

```
import { KyoboNFT__factory }  
  from "../typechain-types";  
  
const nft = KyoboNFT__factory.connect(  
  "0x실제컨트랙트주소",  
  signer  
);  
await nft.mintBatch(to, ids, amounts, "0x");
```

Factory.connect() = 타입 안전한 컨트랙트 연결.

ABI 변경 → 컴파일 에러 연쇄 — TypeChain의 핵심 가치

KyoboNFT.sol의 mintBatch 시그니처를 변경했다고 가정



멀티레포였다면?

배포 후 런타임에 발견. 모노레포 + TypeChain = 컴파일 단계에서 100% 차단.

M1 전체 정리 — S1~S4 핵심 개념

S	주제	핵심 개념	기억할 것
S1	선행 과정 매핑	ERC-20 → ERC-1155 / REST API → TX 상태머신	이 과정의 출발점
S2	5레이어 아키텍처	사용자앱→DMZ→내부망→VASP→블록체인 / IBlockchainAdapter	Phase 1 전체 구조
S3	스켈레톤 구조	모노레포 / Dependency Rule / Strategy Pattern	코드 읽는 법
S4	개발환경 + Fork	HH Network vs Fork / .env / config.ts / TypeChain	실습 기반

S4 핵심 3원칙

① 환경 구분

로컬 노드 → 개발·테스트
Sepolia → 배포 검증
[맛보기] Fork → VASP
통합
M6~M7에서 집중 사용

② .env 분리

.env = 커밋 금지
.env.example = 커밋 필수
API 키 절대 코드에
넣지 말 것

③ TypeChain

컴파일 = 타입 자동 생성
ABI 변경 → 컴파일 에러
런타임 에러 → 빌드 에러

환경 구분: 정확도 / .env: 보안 / TypeChain: 타입 안전성

M1 완료 기준 — 최종 확인

아래 항목 모두 확인 후 M2로 진입

- 1 의존성 설치 완료 (npm install 에러 없음)
- 2 .env 파일 생성 + SEPOLIA_RPC_URL / DEPLOYER_PRIVATE_KEY 입력 완료
- 3 hardhat.config.ts 구성 완료 (forking 없음 — 로컬 노드 기본)
- 4 npx hardhat node — 로컬 노드 기동 성공 + 계정 목록 출력
- 5 npx hardhat compile — 타입 에러 없음 + typechain-types/ 생성
- 6 npx hardhat test — failing 없음, pending 목록 파악
- 7 5레이어 설명 가능 (사용자앱→DMZ→내부망→VASP→블록체인)
- 8 Dependency Rule 설명 가능 (역방향 = 순환 참조)

M2 미리보기 — DMZ 이벤트 파이프라인

다음: M2 S5~S12 — DMZ 이벤트 파이프라인 (10세션 10시간)

S	주제
S5	Webhook 즉시 처리의 위험 + 202 패턴 설계
S6	Redis Streams 내부 구조 + At-least-once 보장 원리
S7	Redis Streams CLI 실습 + 미ACK 재처리 시뮬레이션
S8	Webhook 보안 검증 + Consumer Group 병렬 처리
S9	이벤트 소비 순서 불변 규칙 + At-least-once 설계
S10	EventConsumer 구현 + 멍등 처리 통합 검증
S11	Dead Letter Queue 설계 + 운영
S12	Finalized 블록 기준 처리 + 파이프라인 장애 복원

S4 · M1 완료

M1 완료 — 교보 디지털 자산 플랫폼 개발 환경 구성 완료

로컬 노드
기동 확인
✓ 완료

컴파일
TypeChain 생성
✓ 완료

테스트
pending 확인
M6에서 채울 것

다음 세션: M2 S5 — Webhook 즉시 처리의 위험과 202 패턴